Programming Assignment 4:
Adjoint Methods

Mech 588

Due date: April 28

This assignment will explore some, though not nearly all, of the interesting uses of adjoint methods. The primal problem will be a Poisson problem in a square $[0,1]\times[0,1]$, with homogeneous Dirichlet boundary conditions on three sides ($x = 0$, $y = 0$, and $x = 1$); we'll call the solution to the primal problem u and the solution to the adjoint problem v. Other details will vary for different cases. You can use any Poisson solver you have lying around (from 510, for example, or from the multigrid assignment) or write one from scratch.

1. Warm-up problem. (Easy) For this case, the boundary condition at $y = 1$ is $u = 0$ and the source term for the primal problem is $f = -\frac{\pi^3}{4}\sin\pi x \sin\pi y$. For the dual problem, we also have homogeneous Dirichlet boundary conditions, with source term $g = \frac{\pi^5}{2}x(1-x)y(1-y)$.
   The output functional in this case is

   $$J = (u,g) = (v,f)$$

   Solve both the primal and dual problems, and demonstrate that you get the same value for the (discrete) output functional evaluated both ways, and that the discrete output functional converges to its exact value at the order of accuracy of your numerical scheme.

   To start, it would be best to calculate the functional analytically. In order to do this, we need the exact solution to the Poisson problem. We can determine this by inspection, or by using a change of variables. Let's look at the exact solution to the primal problem, u. We notice that the source term in this case is

   $$f = \frac{-\pi^3}{4}\sin(\pi x)\sin(\pi y)$$

   The Poisson problem would be,

   $$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \frac{-\pi^3}{4}\sin(\pi x)\sin(\pi y)$$

   We only have 2nd derivatives on the left side. For an arbitrary sin function,

   $$h(\theta) = \sin(\pi\theta)$$

   the 2nd derivative is given by,

$$\frac{\partial^2 h}{\partial \theta^2} = -\pi^2 \sin(\pi\theta)$$

Taking this, we can deduce that the exact solution of the Poisson problem must have the form,

$$u = \text{asin}(\pi x)\sin(\pi y)$$

We can take the 2nd derivative of this function, with respect to x and y to obtain,

$$\frac{\partial^2 u}{\partial x^2} = -a\pi^2 \sin(\pi x)\sin(\pi y)$$
$$\frac{\partial^2 u}{\partial x^2} = -a\pi^2 \sin(\pi x)\sin(\pi y)$$

We can plug this into the original problem to obtain,

$$-a\pi^2 \sin(\pi x)\sin(\pi y) - a\pi^2 \sin(\pi x)\sin(\pi y) = \frac{-\pi^3}{4}\sin(\pi x)\sin(\pi y)$$

We can rearrange as follows,

$$-2a\pi^2(\sin(\pi x)\sin(\pi y)) = \frac{-\pi^3}{4}\sin(\pi x)\sin(\pi y)$$

Dividing both sides by $\sin(\pi x)\sin(\pi y)$ yields,

$$-2a\pi^2 = -\frac{\pi^3}{4}$$

We can solve for a to obtain,

$$a = \frac{\pi}{8}$$

Yielding an exact solution,

$$u = \frac{\pi}{8}\sin(\pi x)\sin(\pi y)$$

Now, we can calculate the functional as follows.

$$J = (u, g) = \int_{\Omega} u(x, y) * g(x, y)\, dA$$

$$J = \int_0^1 \int_0^1 \frac{\pi}{8}\sin(\pi x)\sin(\pi y)\frac{\pi^5}{2}x(1-x)y(1-y)dxdy$$

Pulling out all of the constants,

$$J = \frac{\pi^6}{16}\int_0^1\int_0^1 \sin(\pi x)\sin(\pi y)\, x(1-x)y(1-y)dxdy$$

We notice that this can be separated into two separate functions, w(x)c(y). We can separate the double integral into the product of two single integrals,

$$J = \frac{\pi^6}{16}\int_0^1 \sin(\pi x)x(1-x)dx \int_0^1 \sin(\pi y)y(1-y)dy$$

We notice that both of the single integrals are the same, just with different variables. We can use this property to rewrite the functional as follows,

$$J = \frac{\pi^6}{16} * \left[\int_0^1 \sin(\pi\gamma)\,\gamma(1-\gamma)d\gamma\right]^2$$

Now, we only have to solve 1 single integral rather than 1 double integral to calculate the functional. Using integration by parts, and applying linearity we can obtain,

$$J = \frac{\pi^6}{16} * \int_0^1 (1-\gamma)\gamma\sin(\pi\gamma)\,d\gamma = \frac{\pi^6}{16} * 2\left[-\frac{(\gamma-1)\gamma\cos(\pi\gamma)}{\pi}\Big|_0^1 + \frac{1}{\pi}\int_0^1 2(\gamma-1)\cos(\pi\gamma)\,d\gamma\right]$$

Again, we can apply integration by parts to obtain,

$$J = \frac{\pi^6}{16} * \left[-\frac{(\gamma-1)\gamma\cos(\pi\gamma)}{\pi}\Big|_0^1 + \frac{1}{\pi}\left\{\frac{2(\gamma-1(\sin(\pi\gamma))}{\pi}\Big|_0^1 - \frac{2}{\pi}\int_0^1 \sin(\pi\gamma)\,d\gamma\right\}\right]$$

Finally, using u-substitution, we can obtain,

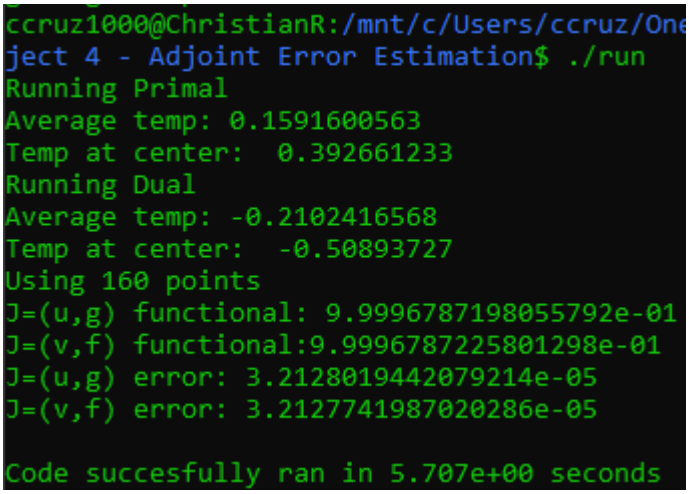
$$J = \frac{\pi^6}{16} * \left[-\frac{(\gamma-1)\gamma\cos(\pi\gamma)}{\pi}\Big|_0^1 + \frac{1}{\pi}\left\{\frac{2(\gamma-1(\sin(\pi\gamma))}{\pi}\Big|_0^1 + \frac{2}{\pi^2}\cos(\pi\gamma)\Big|_0^1\right\}\right]$$

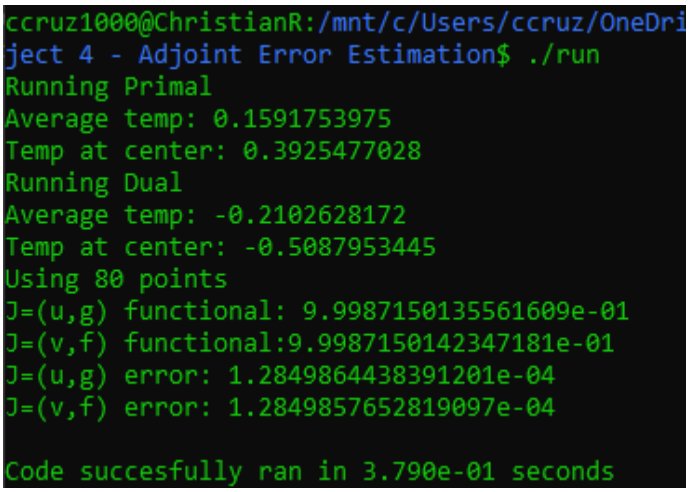
Evaluating the part within the brackets at γ=1 yields, $\frac{2}{\pi^3}$ Evaluating the same terms at γ=0 yields, $-\frac{2}{\pi^3}$. Thus, the functional can be evaluated as,

$$J = \frac{\pi^6}{16} * \left[\frac{2}{\pi^3} - \left(-\frac{2}{\pi^3}\right)\right]^2 = \frac{\pi^6}{16} * \left[\frac{4}{\pi^3}\right]^2 = \frac{\pi^6}{16} * \frac{16}{\pi^6} = 1$$

Thus, the solution to the functional for this case is found to be 1. Similar results can be determined by evaluating the functional as J=(v,f), assuming the exact solution to the adjoint problem, v, is known.

To validate this for the discrete adjoint problem, the code provided on Canvas was modified, such that the boundary conditions and source term can be input into the function call. This allowed running of both the adjoint and the primal problem at the same time. A function was then written, which evaluated the functional using both J=(u, g) and J=(v,f). In order to evaluate the integral, 4-point Gauss-quadrature was utilized to determine u, v, f and g within each cell. These values were then multiplied for their respective functionals, and finally the value for each cell was multiplied by the cell area. Taking the summation of these values for each cell, should lead to an equivalent integral. The following screenshots demonstrate the output for the functional using both derivations, for a total of 40, 80, and 160 cells. It can be seen that the functional gives the same result, regardless of the formulation used.

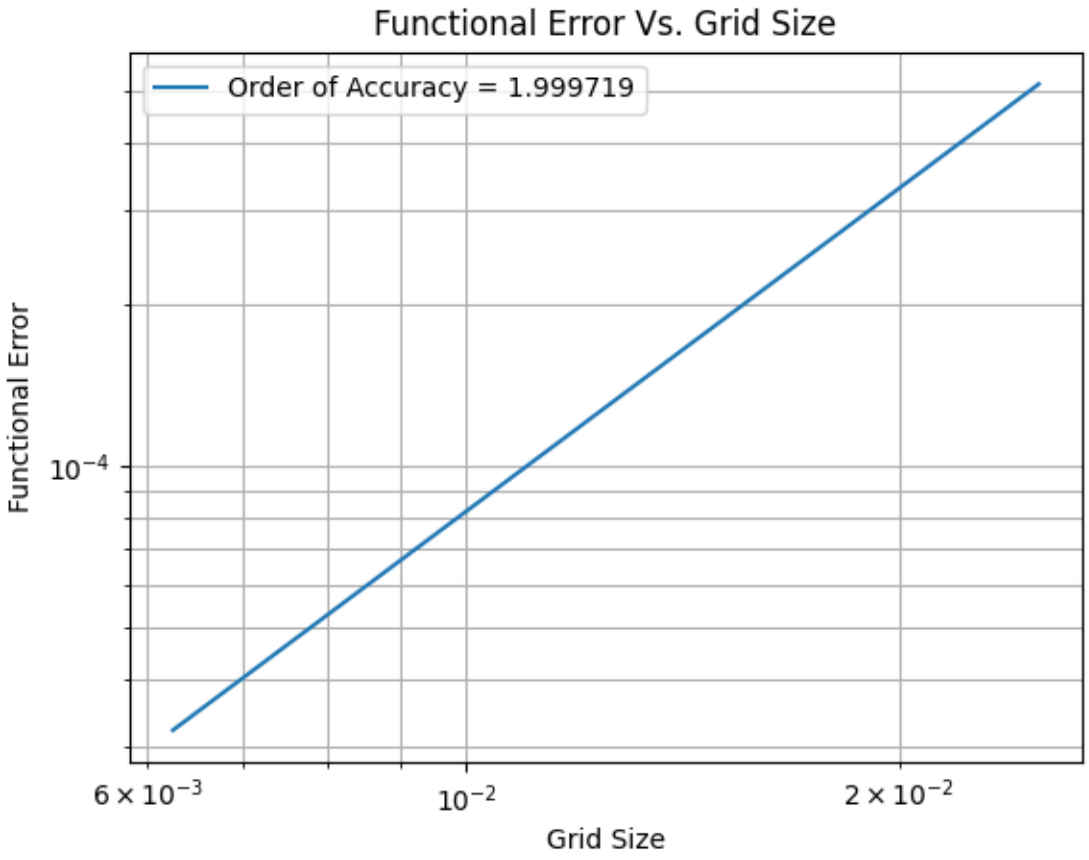




```
ccruz1000@ChristianR:/mnt/c/Users/ccruz/OneD
ject 4 - Adjoint Error Estimation$ ./run
Running Primal
Average temp: 0.1591600563
Temp at center:  0.392661233
Running Dual
Average temp: -0.2102416568
Temp at center:  -0.50893727
Using 160 points
J=(u,g) functional: 9.9996787198055792e-01
J=(v,f) functional:9.9996787225801298e-01
J=(u,g) error: 3.2128019442079214e-05
J=(v,f) error: 3.2127741987020286e-05

Code succesfully ran in 5.707e+00 seconds
```

Finally, the following graph demonstrates the evolution of the error, as the grid spacing is refined.



As can be seen, an order of accuracy of ~2 is found, which is expected, as the Poisson solver being used uses a 2nd order accurate discretization.

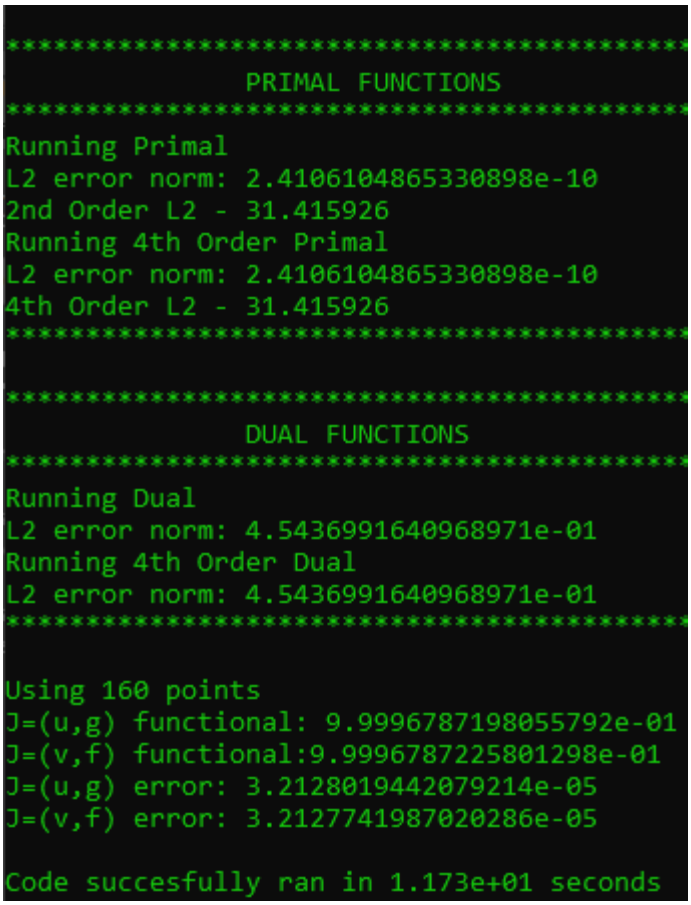
2. Error estimation. (Medium) For the previous case, use a high-order (i.e., more than second order) discretization of the Laplace operator to compute the correction / error estimate term

$$\delta_j = (LU_h - f_h, v_h)$$

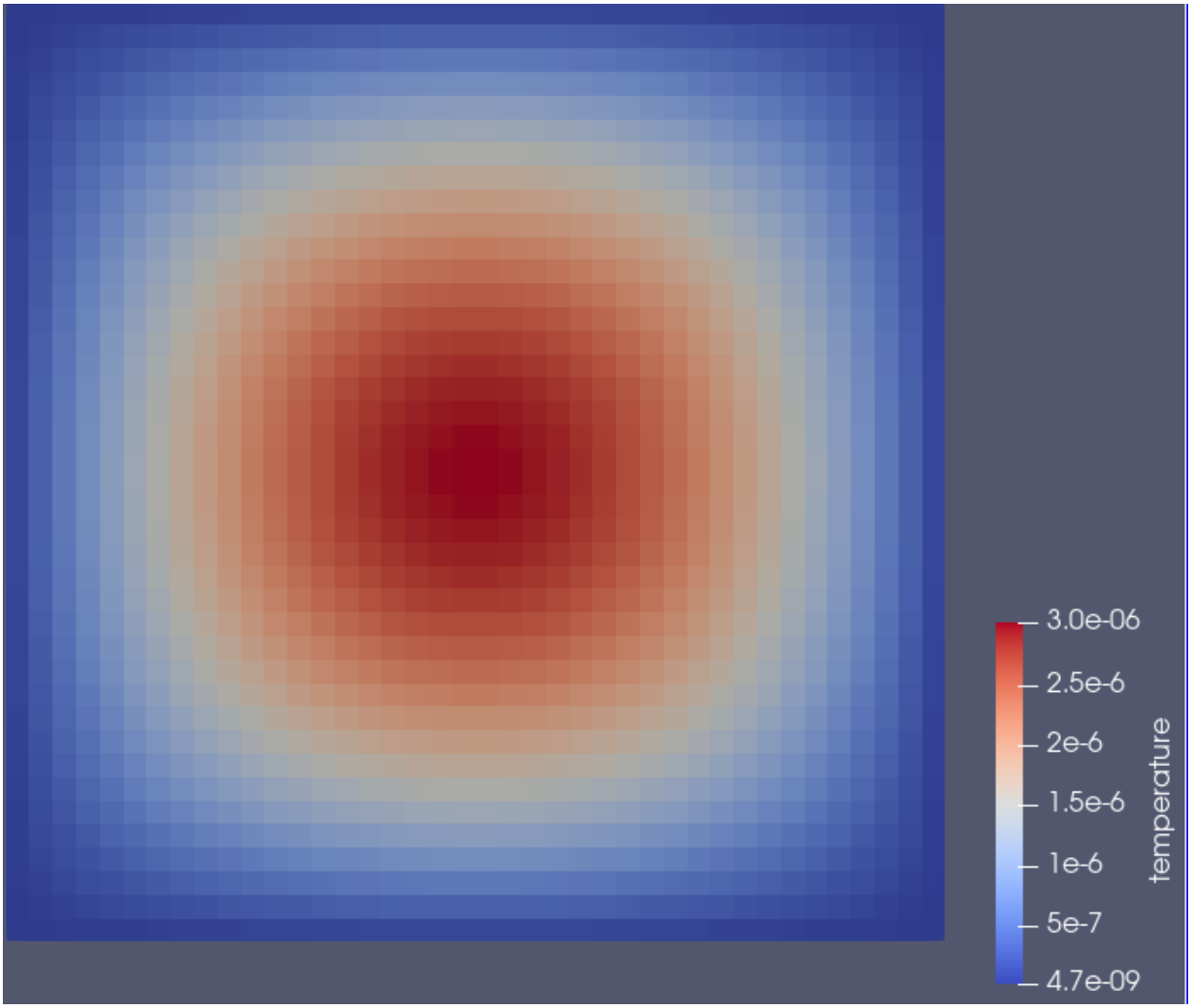
In this case, your high-order operator will fill the role of Lu_h without any need for high order interpolation of the solution.[1] Use your error estimate to get an improved value of the functional J and show that this corrected functional converges with a higher order of accuracy than the uncorrected functional.

In order to compute an error estimate for the functional, we will need a higher order operator. In this case, we can use the 4th order discretization derivation provided on the course website. This discretization uses a 5-point stencil. That is, in order to require the solution, it requires the solution in the current cell, as well as the two preceding cells and the two proceeding cells. This means that a total of 2 ghost cells are needed on each side, in order to maintain the boundary conditions. Unfortunately, all of the solver functions up to this point are coded in a way that they take an array with the number of cells + 2 entries as the input, to allow for one ghost cell on each side. A new series of 4th order functions were written which take the appropriately sized arrays to allow for 2 ghost cells on each side. To ensure the indices for each of these new functions work, the functions were first tested using the same discretization as the 2nd order solver provided. The thought was that if all of the indices and boundary conditions worked properly, the same solution should be obtained for the 2nd order and the 4th order solvers.

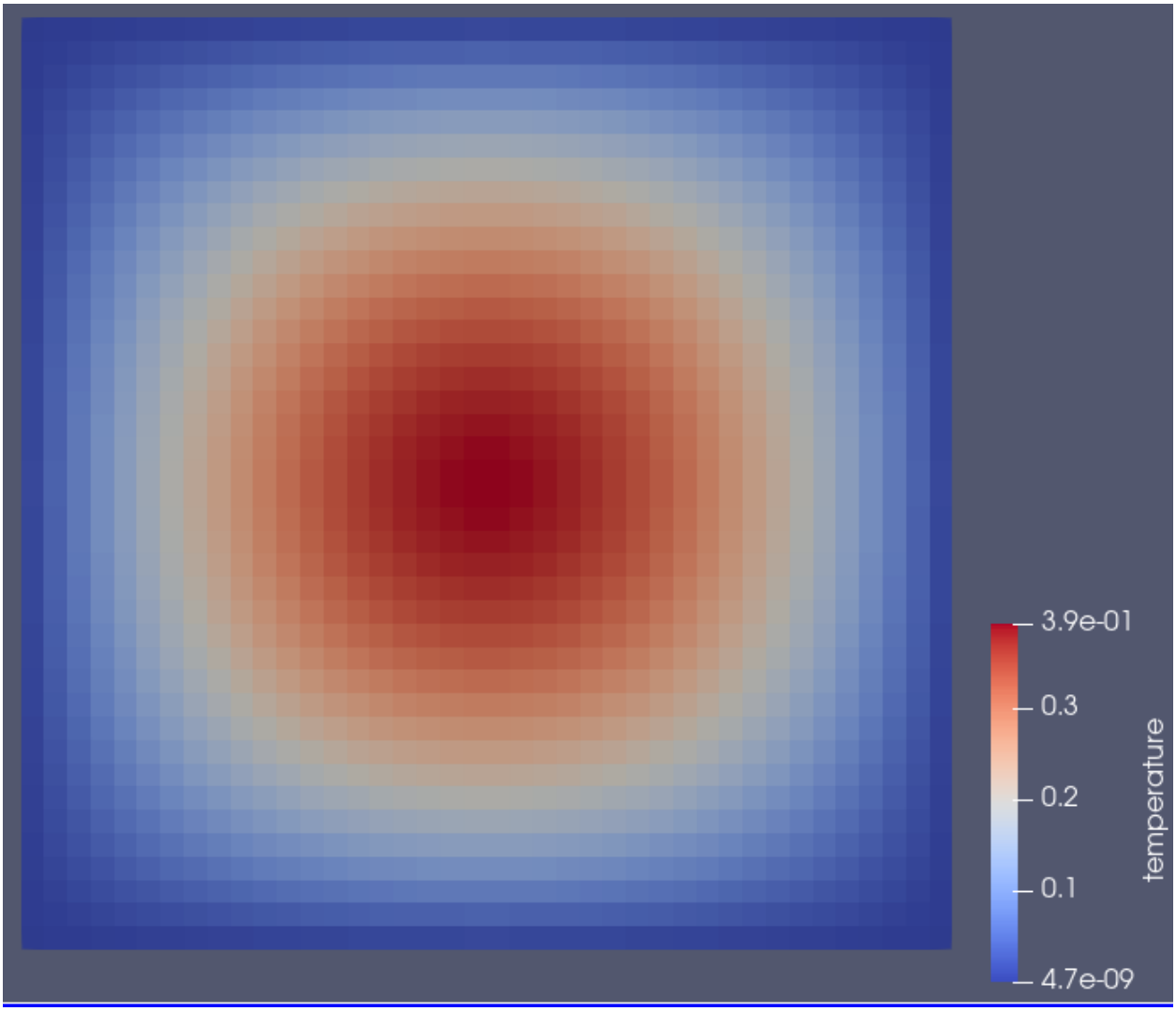
[1] Note that my experiments suggest that this level of approximation, while successful here, isn't sufficient for error estimation for functionals with boundary integrals.



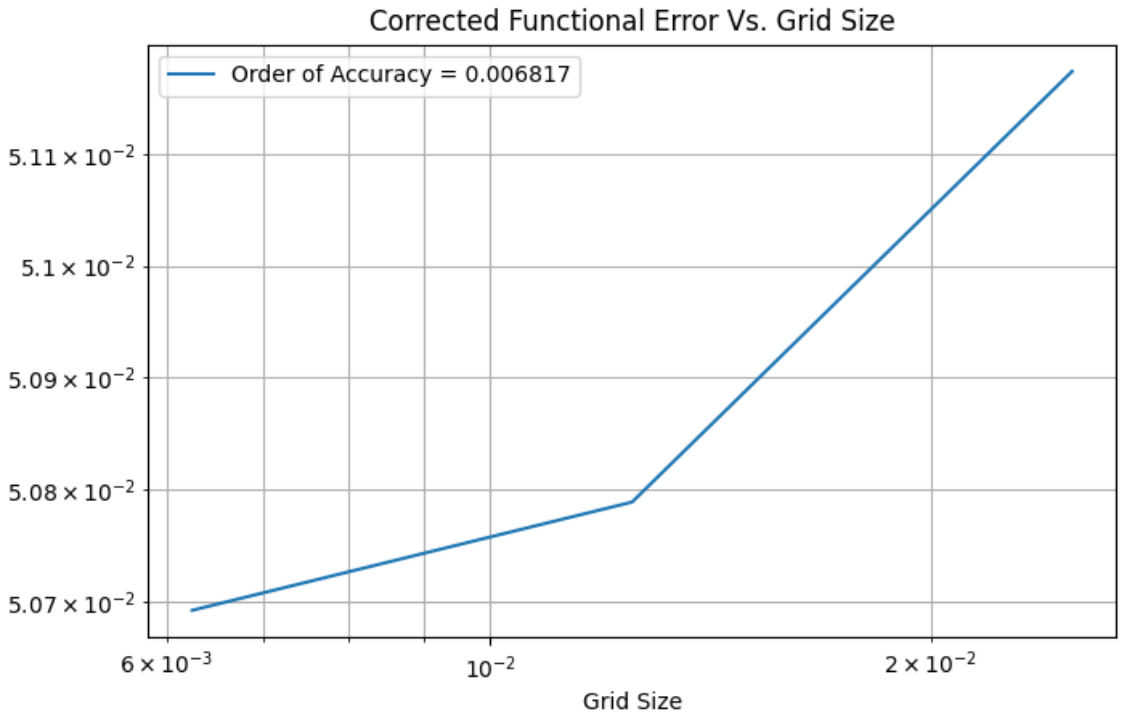
Once this was completed, the boundary conditions and the 4th order discretization. The 4th order discretization for the Poisson equation was taken from the provided derivation on Canvas, while the boundary conditions were taken from a paper found online. [1] A test was performed with a 0 source term to ensure that the homogenous boundary conditions were met. Once this test was passed, the final code was written such that it would compute the solution to the Poisson equation with 4th order spatial accuracy. Unfortunately, a bug still exists in the code, and due to time constraints, it could not be fixed. The solution currently being output can be found below,



The solution computed with the 2nd order solver (which we know works), is



As can be seen, the code vastly underestimates the solution, by about 2 orders of magnitude. It is unclear as to why this bug is occurring. With more time it could surely be found, as it is most likely something small such as an improper index or a sign. Obviously, the functional correction will not work properly if the higher order discretization does not work, but the code for the functional correction was implemented. It is believed that with minimal error, the 4th order implementation could be fixed, and the functional correction should work.



3. Functionals with boundary integrals. (Easy) For this case, use $u = \sin\pi x$ as a boundary condition at $y = 1$; retain the same source terms as in Question 1. Your primal output functional is now

$$J = (u,g)_\Omega + (u_n,h)_{\partial\Omega}$$

where $h = x(1-x)$ on $y = 0$ and zero elsewhere; keep in mind that u_n is defined as the directional derivative in the *outward* normal direction, so $-u_y$ in this case.

It is easy to show that the dual problem in this case has a boundary condition of $v = x(1-x)$ on $y = 0$, and $v = 0$ elsewhere, and that the dual functional is

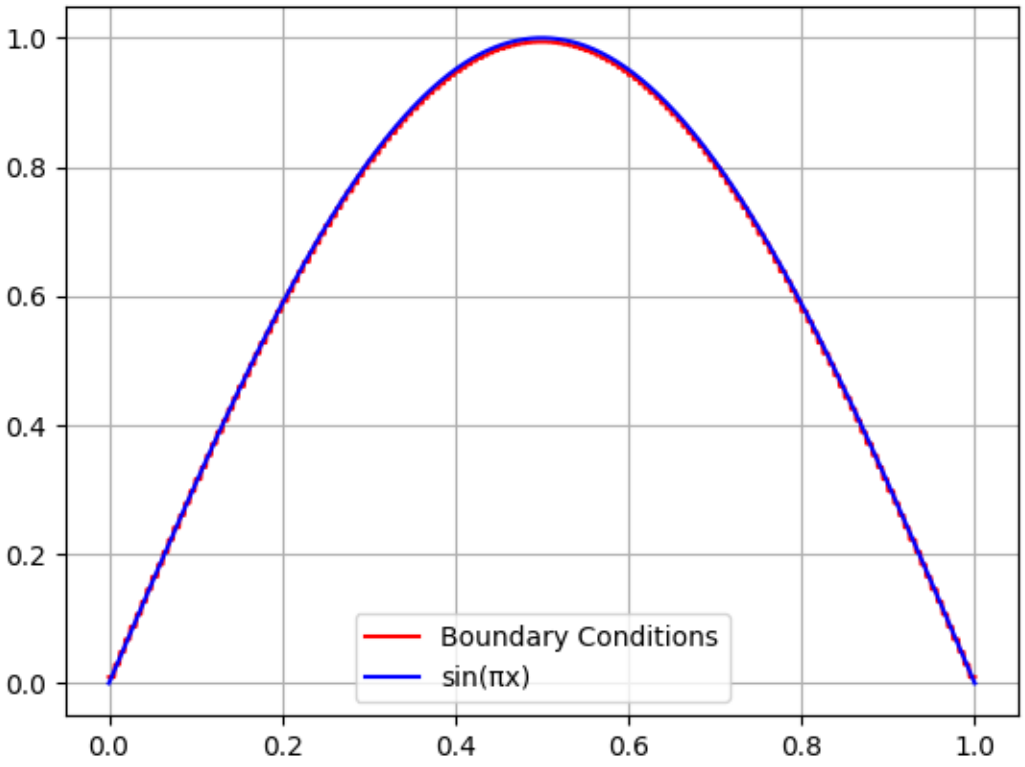
$$J = (v, f)_\Omega + (v_n,e)_{\partial\Omega}$$

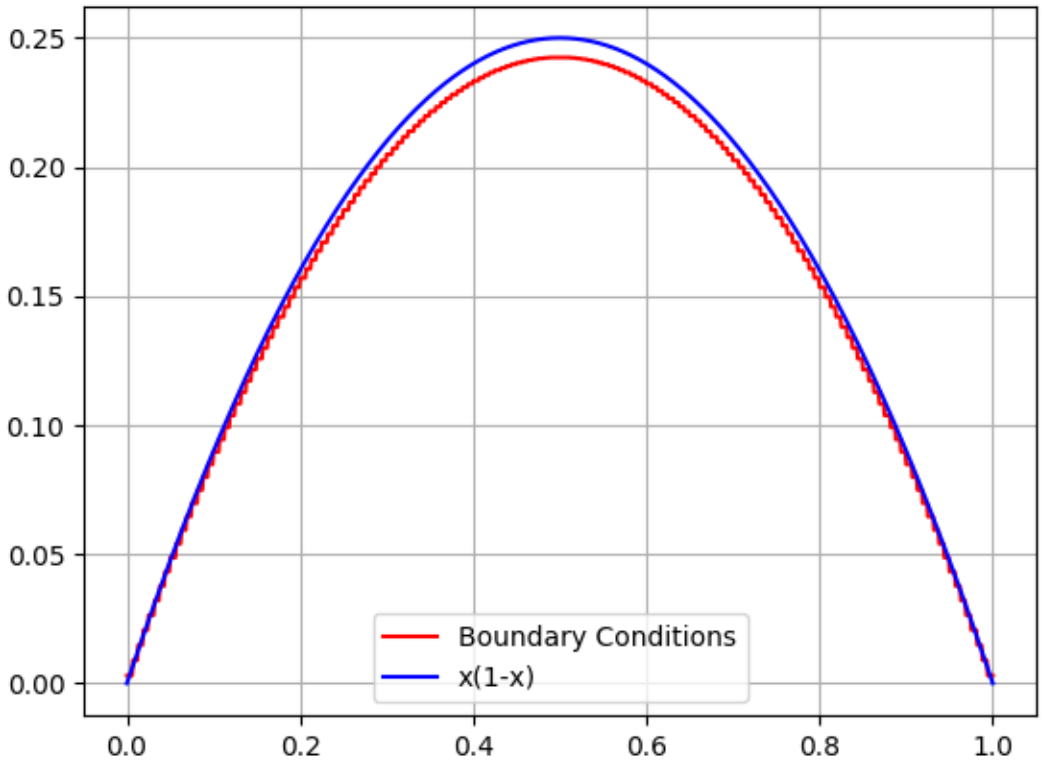
where $e = \sin\pi x$ on $y = 1$ and zero elsewhere.

Compute this functional using both the primal and dual problems, and show that this formulation is adjoint consistent and converges with the correct order of accuracy.

To begin this problem, the first step is to ensure the correct boundary conditions. In order to do this, a .vtk file was saved for both u and g. This was then imported into Paraview, where the “Plot

Over Line" filter was applied to obtain the data at the boundary of interest. The following plots show that the boundary conditions were properly applied.





At the current time of writing I still have a bug in the code for this section. I will submit everything as is, and try to solve it tomorrow. This way if I find the solution I can submit it tomorrow, otherwise I will not lose grades by wasting time since I am out of days.

4. Sensitivity. (Medium to hard) Suppose now that we modify the primal problem from Question 1 so that

$$f = (1+c_1)\left(-\frac{\pi^3}{4}\sin\pi x\sin\pi y\right)+c_2 g$$

In the context of an optimization problem, we would need to be able to compute $\frac{\partial J}{\partial \vec{c}}$. Writing this partial derivative out using the chain rule:

$$J = (u,g)_\Omega = J(u(\sim c))$$

$$\frac{\partial J}{\partial \sim c} = \frac{\partial J}{\partial u}\frac{\partial u}{\partial \vec{c}}$$

(1)

This would require an additional primal solve to estimate the derivative of the solution with respect to each component~c. But we can take advantage of the zero-residual condition:

$$\begin{aligned} L(u) &= f \\ \frac{\partial L}{\partial \vec{c}} \frac{\partial u}{} &= \frac{\partial f}{\partial \vec{c}} \frac{\partial u}{} \\ \frac{\partial u}{\partial \vec{c}} &= \left(\frac{\partial L}{\partial u}\right)^{-1} \left(\frac{\partial f}{\partial \vec{c}}\right) \end{aligned}$$

This doesn't reduce the amount of work yet, but substituting this into Eqn. 1 and taking the transpose does:

$$\left(\frac{\partial J}{\partial \vec{c}}\right)^T = \left(\frac{\partial f}{\partial \vec{c}}\right)^T \left(\frac{\partial L}{\partial u}\right)^{-T} \left(\frac{\partial J}{\partial u}\right)_T$$

where the last two terms represent the solution to the adjoint problem with $\frac{\partial J}{\partial u}$ as a source term. Use the adjoint formulation to find the sensitivity of this objective function for $c_1 = 0$, $c_2 = 0.5$; use finite differences to verify your sensitivity result.